

UNIVERSIDAD SERGIO ARBOLEDA

ESCUELA DE CIENCIAS EXACTAS E INGENIERÍA

CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL

Procesamiento Paralelo y el Problema del Viajero

Presentado por:

Paula A Caballero

Julián E Rincón

Profesor:

Guillermo Andrés De Mendoza

Bogota D.C

2025

Objetivo del Laboratorio

El objetivo de este laboratorio es implementar y comparar la eficiencia de un algoritmo secuencial y uno paralelo (utilizando múltiples procesos) para resolver el Problema del Viajante (Travelling Salesperson Problem, TSP) mediante el método de fuerza bruta. El resultado de este laboratorio será un informe que demuestre el speedup (aceleración) logrado con el paralelismo.

Marco teórico

El problema del viajero (TSP)

El Problema del Viajero (Traveling Salesman Problem, TSP) es uno de los problemas más clásicos y estudiados en la computación y la optimización combinatoria. Se plantea de la siguiente manera: dado un conjunto de n ciudades y las distancias entre cada par de ellas, se busca encontrar el recorrido más corto posible que permita visitar cada ciudad exactamente una vez y regresar al punto de partida.

Matemáticamente, el TSP se modela como un grafo completo y ponderado, donde los vértices representan las ciudades y las aristas, las distancias o costos de desplazamiento. El objetivo es encontrar el ciclo hamiltoniano de costo mínimo dentro de ese grafo.

Complejidad y dificultad del TSP

El TSP pertenece a la clase de problemas NP-duros (NP-hard), lo que significa que no se conoce ningún algoritmo que pueda resolverlo de manera exacta en tiempo polinómico. La cantidad de posibles rutas crece factorialmente con el número de ciudades, es decir, existen $(n-1)! / 2$ rutas posibles para un grafo completamente conectado (considerando simetría).

Por ejemplo, con solo 10 ciudades existen 181,440 rutas posibles, y con 20 ciudades ya son más de 60 billones. Esta explosión combinatoria hace que el TSP sea extremadamente costoso de resolver mediante métodos exactos cuando n es grande.

Por esa razón, el TSP es un problema de referencia (benchmark) en la teoría de la computación, ya que permite estudiar la eficiencia de distintos paradigmas de resolución: desde algoritmos exactos (fuerza bruta, branch and bound, programación dinámica) hasta heurísticas (como el vecino más cercano o el algoritmo genético).

Enfoque de fuerza bruta

El método de fuerza bruta (brute-force) consiste en generar y evaluar todas las posibles permutaciones del recorrido del viajero, calculando la distancia total de cada una y seleccionando la de menor costo. Aunque garantiza la obtención de la solución óptima, su complejidad factorial lo hace impracticable para instancias grandes.

Este enfoque es útil desde el punto de vista académico porque permite comprender los límites del rendimiento computacional. Dado que cada posible ruta se evalúa de forma independiente, el algoritmo presenta una alta granularidad de paralelismo, es decir, cada combinación puede procesarse simultáneamente sin dependencia de las demás.

Paralelismo y TSP como caso de estudio

El TSP por fuerza bruta es un caso de estudio ideal para el análisis de computación paralela por varias razones:

- Independencia de tareas: cada recorrido puede evaluarse de manera autónoma, lo que facilita su distribución entre múltiples procesadores o hilos.
- Escalabilidad natural: al aumentar el número de procesadores, se puede reducir el tiempo total de ejecución casi linealmente (hasta ciertos límites de sincronización).
- Carga balanceada: las operaciones requeridas para evaluar cada ruta son similares, lo que minimiza el desequilibrio de carga entre procesos.
- Simplicidad conceptual: el algoritmo de fuerza bruta es fácil de entender e implementar,

lo que lo convierte en un punto de partida ideal para experimentar con técnicas de paralelismo distribuido (como MPI, OpenMP o CUDA).

Metodología:

Datos

El conjunto de ciudades utilizado se define mediante coordenadas cartesianas (x, y) que representan sus posiciones en el plano.

En este experimento se emplearon 10 ciudades, numeradas del 0 al 9:

CIUDAD	COORDENADA (X, Y)
0	(0, 0)
1	(2, 8)
2	(5, 3)
3	(8, 6)
4	(9, 2)
5	(3, 9)
6	(4, 1)
7	(7, 7)
8	(1, 4)
9	(6, 5)

Estas coordenadas sirven como entrada tanto para el algoritmo secuencial como para el paralelo, permitiendo comparar el rendimiento en condiciones equivalentes. Cada par de ciudades se conecta mediante una distancia euclidiana calculada con la siguiente forma:

$$d(i, j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

Configuración del Hardware

El código obtiene automáticamente la configuración del sistema mediante las librerías **platform** y **psutil**.

```
=====
INFORMACIÓN DEL SISTEMA
=====

PROCESADOR:
  Procesador: AMD64 Family 23 Model 104 Stepping 1, AuthenticAMD
  Arquitectura: AMD64
  Núcleos físicos: 8
  Núcleos lógicos (threads): 16
  Frecuencia actual: 1801.00 MHz
  Frecuencia máxima: 1801.00 MHz

MEMORIA RAM:
  Total: 15.34 GB
  Disponible: 4.24 GB
  En uso: 11.10 GB
  Porcentaje usado: 72.4%

SISTEMA OPERATIVO:
  Sistema: Windows
  Versión: 10.0.26100
  Release: 10

ENTORNO DE PYTHON:
  Versión de Python: 3.11.1
  Implementación: CPython
```

Configuración del Software}

Elemento	Versión / Descripción
Lenguaje	Python 3.10.12
Librerías utilizadas	itertools, math, time, multiprocessing, platform, psutil, typing
Tipo de ejecución	Script local (main.py) ejecutado desde consola
Sistema operativo de prueba	Windows 10 (x64)

Todas las librerías utilizadas son parte de la biblioteca estándar de Python, excepto psutil, que se emplea para medir información del sistema y uso de memoria.

Explicación de los algoritmos desarrollados.

Cálculo de distancias

Se utiliza la distancia euclidiana entre las coordenadas de las ciudades para determinar el costo de cada tramo.

La función **calcular_distancia_ruta()** evalúa la distancia total de una ruta completa, incluyendo el regreso a la ciudad inicial, y realiza múltiples verificaciones de validez (segmentos negativos, infinitos o repetidos).

Algoritmo Secuencial (Fuerza Bruta)

Función: `tsp_secuencial(ciudades)`

Recorre todas las permutaciones posibles de las ciudades (exceptuando la inicial).

Calcula la distancia total de cada ruta con `calcular_distancia_ruta()`.

Conserva la ruta de menor distancia total.

Su complejidad temporal es factorial:

$$O((n - 1)!)$$

lo que lo hace muy costoso para grandes valores de n .

Ventajas:

- Implementación sencilla.
- Garantiza encontrar la solución óptima.

Desventajas:

- Escalabilidad extremadamente limitada.
- El tiempo de ejecución crece exponencialmente con el número de ciudades.

Algoritmo Paralelo (Fuerza Bruta Distribuida)

Función: `tsp_paralelo(ciudades, num_procesos)`

Divide el conjunto total de rutas entre varios procesos usando la librería `multiprocessing`.

Cada proceso evalúa un subconjunto distinto de rutas mediante la función auxiliar

`evaluar_grupo_rutas()`.

Al finalizar, los resultados de todos los procesos se combinan para obtener la mejor ruta global.

Este enfoque busca acelerar la exploración de todas las permutaciones aprovechando múltiples núcleos de CPU.

Ventajas:

- Reduce considerablemente el tiempo total de ejecución.
- Permite aprovechar el paralelismo del hardware.

Desventajas:

- Mayor consumo de memoria.
- Overhead de sincronización y comunicación entre procesos.
- Limitado por el número de núcleos disponibles.
- Comparación de rendimiento
- El programa mide automáticamente:
- Tiempo de ejecución de ambos métodos.
- Uso de memoria antes y después.

Speedup:

$$S = \frac{T_{secuencial}}{T_{paralelo}}$$

Eficiencia paralela:

$$E = \frac{S}{N} \times 100$$

donde N es el número de procesos utilizados.

Se presentan métricas como rutas evaluadas, porcentaje de mejora y tiempo ahorrado.

Resultados

Métrica	Secuencial	Paralelo
Tiempo de ejecución (s)	16.07	3.74
Rutas evaluadas	362,880	362,880
Procesos utilizados	1	16
Distancia mínima	32.5078	32.5078
Speedup	–	4.30x
Eficiencia paralela	–	26.85%
Ahorro de tiempo	–	12.33 s

El análisis comparativo entre los algoritmos secuencial y paralelo evidencia una mejora significativa en el tiempo de ejecución al aplicar técnicas de procesamiento paralelo.

El algoritmo secuencial evaluó 362,880 rutas en 16.07 segundos, utilizando un solo núcleo del procesador. En contraste, la versión paralela distribuyó la misma cantidad de rutas entre 16 procesos, logrando una ejecución de solo 3.74 segundos, lo que representa una reducción del 76.7% en el tiempo total.

El speedup alcanzado fue de 4.30x, indicando que la versión paralela fue más de cuatro veces más rápida que la secuencial. Sin embargo, la eficiencia paralela fue del 26.85%, lo cual sugiere que no se aprovechó completamente la capacidad teórica de los 16 núcleos. Esto se debe al overhead asociado con la creación y coordinación de procesos, la división de tareas y la comunicación entre ellos.

A pesar de este costo adicional, el paralelismo demostró una clara ventaja en el procesamiento de tareas altamente independientes, como la evaluación de rutas en el TSP mediante fuerza bruta. Además, ambos algoritmos hallaron la misma solución óptima (distancia mínima de 32.5078 unidades), confirmando que la paralelización no afectó la precisión del resultado.

Conclusiones

La experimentación con procesamiento paralelo permitió comprobar de manera práctica cómo la distribución de tareas entre múltiples núcleos puede mejorar significativamente el rendimiento computacional en problemas intensivos en cálculo. En este caso, el algoritmo paralelo logró reducir el tiempo de ejecución a menos de una cuarta parte del tiempo requerido por la versión secuencial.

Este ejercicio evidencia que:

El paralelismo es especialmente útil en algoritmos que pueden dividir su carga de trabajo en

tareas independientes, como ocurre con la evaluación de rutas en el TSP.

Sin embargo, la eficiencia no crece de forma lineal con el número de procesos, ya que existen limitaciones derivadas de la sincronización, la comunicación y la sobrecarga de creación de procesos.

A pesar de estas limitaciones, la ganancia en velocidad y aprovechamiento del hardware es significativa y justifica la implementación de técnicas paralelas en problemas de gran escala.

En conclusión, la aplicación práctica del procesamiento paralelo no solo reduce los tiempos de ejecución, sino que también fomenta una mejor comprensión de cómo optimizar los recursos del sistema en la resolución de problemas complejos. Este caso demuestra cómo el paralelismo puede transformar un enfoque ineficiente (fuerza bruta) en una solución computacionalmente viable para escenarios pequeños o medianos del TSP.